

# Nginx Load Balancer + Ngrok Public Tunnel

Complete step-by-step guideline

Three web servers (Phase 1, 2, 3) on one public address



Covers: Ubuntu/Debian Linux (native install) and Docker Compose (Windows/macOS/Linux)

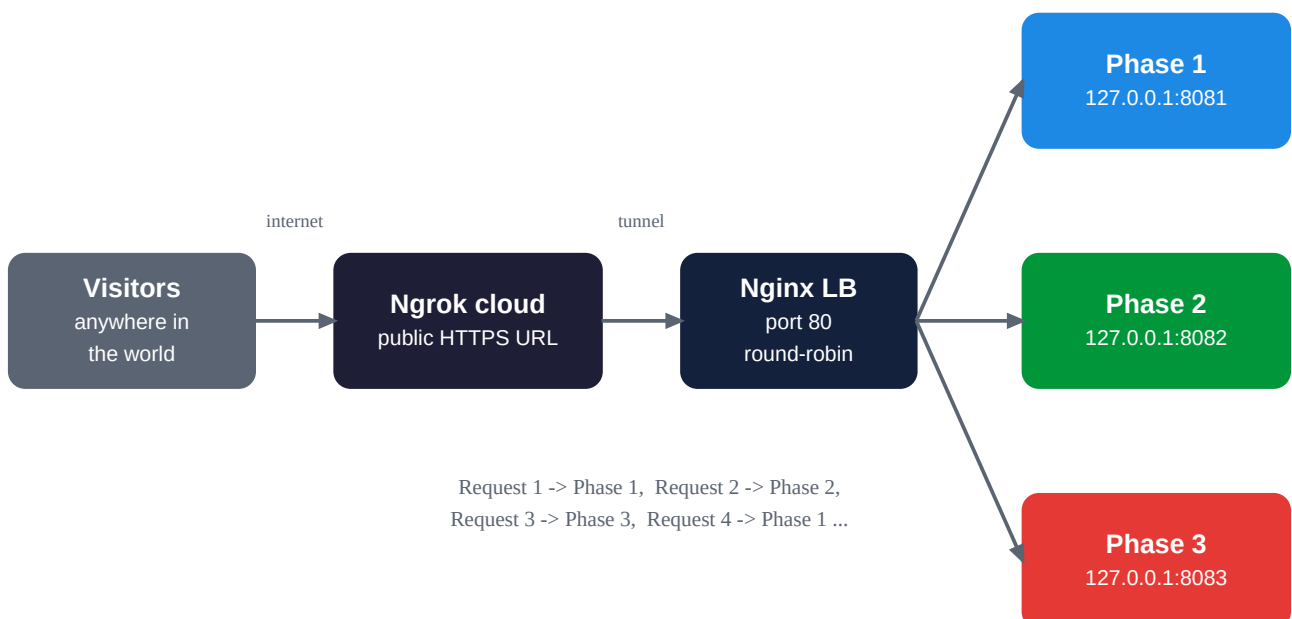
Estimated time: 20-30 minutes

# Contents

| Section                 | What you will do                                        |
|-------------------------|---------------------------------------------------------|
| 1. Overview             | Understand the architecture and how round-robin works   |
| 2. Prerequisites        | What you need before starting                           |
| Step 1                  | Install Nginx                                           |
| Step 2                  | Create the three web servers (Phase 1, 2, 3)            |
| Step 3                  | Configure Nginx as a load balancer                      |
| Step 4                  | Test the alternation locally                            |
| Step 5                  | Install Ngrok and expose the load balancer              |
| Step 6                  | Share the link and verify from outside                  |
| 7. Optional tuning      | Weights, health checks, other algorithms                |
| 8. Troubleshooting      | Fixes for the most common problems                      |
| 9. Cleanup and security | Shut everything down safely                             |
| Appendix A              | Docker Compose alternative (works on Windows and macOS) |

## 1. Overview

The goal is one public web address that shows a different page each time it is refreshed. Three separate web servers each serve a page labelled **Phase 1**, **Phase 2** or **Phase 3**. Nginx sits in front of them as a **load balancer** and hands each incoming request to the next server in turn. Ngrok then publishes the Nginx port to the internet through a secure tunnel, so you do not need a public IP address or router port-forwarding.



## How it works

- **Web servers.** Three small Nginx server blocks listening on 127.0.0.1 ports 8081, 8082 and 8083, each with its own HTML page.
- **Load balancer.** One Nginx server block on port 80 with an `upstream` group. The default method is **round-robin**: request 1 goes to server 1, request 2 to server 2, request 3 to server 3, then it starts over.
- **Ngrok.** A small agent on your machine opens an outbound connection to Ngrok's cloud and returns a public HTTPS URL that forwards to port 80.

### Note on what visitors will see

Round-robin alternates *requests*, not visitors. If many people use the link at the same time, they share the same rotation, so one person may see Phase 1 then Phase 3 because someone else received Phase 2 in between. Testing alone gives a clean 1, 2, 3, 1, 2, 3 pattern.

## 2. Prerequisites

| Requirement      | Details                                                                                                             |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| Operating system | Ubuntu 20.04+ or Debian 11+ (native steps). Windows users can use WSL2 (Ubuntu) or follow Appendix A (Docker).      |
| Privileges       | A user with <code>sudo</code> rights                                                                                |
| Internet access  | Needed to install packages and for the Ngrok tunnel (outbound only; no firewall changes required)                   |
| Ngrok account    | Free account at <a href="https://ngrok.com">ngrok.com</a> . You will copy your <b>authtoken</b> from the dashboard. |
| Free ports       | 80, 8081, 8082, 8083 must not be used by another program                                                            |

### STEP 1

## Install Nginx

Update the package list, install Nginx and confirm it is running.

```
sudo apt update
sudo apt install -y nginx curl
sudo systemctl enable --now nginx
nginx -v
```

Open `http://localhost` in a browser or run `curl -I http://localhost`. You should see the default "Welcome to nginx!" page (HTTP 200). That page comes from a default site that we will remove in Step 3.

### macOS or other systems

On macOS use `brew install nginx`; config files then live under `/opt/homebrew/etc/nginx/` (or `/usr/local/etc/nginx/` on Intel Macs) and you would place the server blocks there. If you prefer not to adapt paths, use Appendix A.

### STEP 2

## Create the three web servers

### 2.1 Create the Phase pages

This loop creates three folders and a colour-coded HTML page in each: blue for Phase 1, green for Phase 2 and red for Phase 3.

```

sudo mkdir -p /var/www/phase1 /var/www/phase2 /var/www/phase3
COLORS=("1e88e5" "43a047" "e53935")

for i in 1 2 3; do
  C=${COLORS[${i}-1]}
  sudo tee /var/www/phase${i}/index.html > /dev/null <<EOF
<!DOCTYPE html>
<html lang="en"><head><meta charset="utf-8">
<title>Phase ${i}</title>
<style>
body{margin:0;height:100vh;display:flex;align-items:center;
justify-content:center;font-family:Arial,sans-serif;
background:$C;color:#fff;text-align:center}
h1{font-size:7rem;margin:0}
</style></head>
<body><div><h1>PHASE ${i}</h1>
<p>Served by web server ${i} (port 808${i})</p></div></body></html>
EOF
done

```

## 2.2 Define the three backend servers

Create `/etc/nginx/conf.d/phase-backends.conf` with `sudo nano` and paste the following. Each server listens on `127.0.0.1` only, so the pages can be reached solely through the load balancer. The X-Backend header lets you see which server answered.

```

# /etc/nginx/conf.d/phase-backends.conf
server {
    listen 127.0.0.1:8081;
    server_name _;
    root /var/www/phase1;
    index index.html;
    add_header X-Backend "phase1" always;
}
server {
    listen 127.0.0.1:8082;
    server_name _;
    root /var/www/phase2;
    index index.html;
    add_header X-Backend "phase2" always;
}
server {
    listen 127.0.0.1:8083;
    server_name _;
    root /var/www/phase3;
    index index.html;
    add_header X-Backend "phase3" always;
}

```

### STEP 3

## Configure Nginx as a load balancer

### 3.1 Remove the default site

The default site also listens on port 80 and would conflict with the load balancer.

```
sudo rm -f /etc/nginx/sites-enabled/default
```

### 3.2 Create the load balancer configuration

Create `/etc/nginx/conf.d/load-balancer.conf`:

```
# /etc/nginx/conf.d/load-balancer.conf
upstream phase_servers {
    # Round-robin is the default method
    server 127.0.0.1:8081;
    server 127.0.0.1:8082;
    server 127.0.0.1:8083;
}

server {
    listen 80 default_server;
    server_name _;

    # Browsers ask for /favicon.ico on every visit. Without this rule each
    # favicon request consumes one round-robin turn and the pages seem to skip.
    location = /favicon.ico {
        access_log off;
        return 204;
    }

    location / {
        proxy_pass http://phase_servers;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Stop browsers and proxies from caching a page, so every
        # refresh reaches the load balancer and moves to the next phase
        add_header Cache-Control "no-store, no-cache, must-revalidate" always;
    }
}
```

### 3.3 Validate and reload

```
sudo nginx -t
sudo systemctl reload nginx
```

The test should print `syntax is ok` and `test is successful`. If it reports an error, read the file name and line number it shows and correct that line.

#### STEP 4

### Test the alternation locally

Send six requests and print which phase answered each one:

```
for i in $(seq 1 6); do curl -s http://localhost | grep -o "PHASE [0-9]"; done
```

Expected output (the cycle repeats):

```
PHASE 1
PHASE 2
PHASE 3
PHASE 1
PHASE 2
PHASE 3
```

You can also check the header of a single response and confirm each backend directly:

```
curl -sI http://localhost | grep -i x-backend
curl -s http://127.0.0.1:8081 | grep -o "PHASE [0-9]"
curl -s http://127.0.0.1:8082 | grep -o "PHASE [0-9]"
curl -s http://127.0.0.1:8083 | grep -o "PHASE [0-9]"
```

Finally open `http://localhost` in a browser and press refresh repeatedly. The colour and label should rotate blue, green, red. Do not continue until this works locally.

**Checkpoint**

If the local test shows only one phase or an error, fix it now using Section 8 before starting Ngrok.

**STEP 5****Install Ngrok and expose the load balancer**

### 5.1 Create an account and get your authtoken

- Sign up at **ngrok.com** and log in to the dashboard.
- Open the **Getting Started / Setup** page and copy your authtoken. Treat it like a password.

### 5.2 Install the Ngrok agent

The dashboard's setup page shows the current install command for your operating system; use that one because repositories and download links change over time. Common options:

| System          | Install method                                                                                                                                                                                             |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ubuntu / Debian | Follow the apt repository commands on the dashboard setup page, or download the Linux archive from <a href="https://ngrok.com/download">ngrok.com/download</a> and move the ngrok binary to /usr/local/bin |
| macOS           | <code>brew install ngrok</code>                                                                                                                                                                            |
| Windows         | Download from <a href="https://ngrok.com/download">ngrok.com/download</a> or use the Microsoft Store / winget / Chocolatey                                                                                 |

Check the installation:

```
ngrok version
```

### 5.3 Add your authtoken (one time)

```
ngrok config add-authtoken YOUR_AUTHTOKEN_HERE
```

### 5.4 Start the tunnel

Point Ngrok at the **load balancer port (80)**, not at one of the backends:

```
ngrok http 80
```

Ngrok shows a status screen similar to this:

```
Session Status   online
Web Interface    http://127.0.0.1:4040
Forwarding       https://a1b2-c3d4.ngrok-free.app -> http://localhost:80
```

The **Forwarding** address ending in `.ngrok-free.app` (or another Ngrok domain) is your public link. Keep this terminal window open: closing it stops the tunnel.

**Optional: a stable address**

Free random URLs change every time you restart Ngrok. In the dashboard under Domains you can claim a static domain and start the tunnel with `ngrok http --url=YOUR-DOMAIN.ngrok-free.app 80`. Domain availability and limits depend on your plan, so check the dashboard.

**STEP 6****Share the link and verify from outside**

- Copy the https Forwarding URL and open it on a device that is **not** on your Wi-Fi (for example a phone using mobile data) to prove it is truly public.

- Free Ngrok accounts show a one-time **warning page** to browser visitors. Clicking **Visit Site** continues to your pages; this is normal.
- Refresh repeatedly. The pages should rotate Phase 1, Phase 2, Phase 3 on the same address.
- Watch the traffic live at `http://127.0.0.1:4040` (the Ngrok inspector). Each refresh appears as a request.

You can also test from any terminal. The extra header skips the browser warning page:

```
for i in $(seq 1 6); do
  curl -s -H "ngrok-skip-browser-warning: 1" https://YOUR-URL.ngrok-free.app \
    | grep -o "PHASE [0-9]"
done
```

## 7. Optional tuning

Change these inside the upstream `phase_servers { ... }` block, then run `sudo nginx -t && sudo systemctl reload nginx`.

| Goal                                               | Configuration                                                                                                            |
|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Weighted rotation (Phase 1 gets twice the traffic) | <code>server 127.0.0.1:8081 weight=2;</code>                                                                             |
| Send to the least busy server                      | Add <code>least_conn</code> ; as the first line in the upstream block                                                    |
| Sticky visitors (same person keeps the same phase) | Add <code>ip_hash</code> ; . This <b>stops</b> a single visitor from seeing alternation, so do not use it for this demo. |
| Automatic failover                                 | <code>server 127.0.0.1:8081 max_fails=2 fail_timeout=10s;</code> on each server line                                     |
| Mark a server as spare                             | <code>server 127.0.0.1:8083 backup;</code>                                                                               |

Failover demo: stop one backend by removing its server block (or its port), reload Nginx, and refresh. Requests keep working and only the remaining phases appear. With `max_fails` set, Nginx skips a failed server automatically.

## 8. Troubleshooting

| Symptom                                      | Likely cause and fix                                                                                                                                                |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Welcome to nginx page instead of Phase pages | Default site is still enabled. Run <code>sudo rm -f /etc/nginx/sites-enabled/default</code> , then reload.                                                          |
| nginx -t: duplicate default_server           | Same cause as above. Only one server block may use default_server on port 80.                                                                                       |
| 502 Bad Gateway                              | A backend is not listening. Check <code>sudo ss -tlnp   grep 808</code> ; all three ports should appear. Confirm phase-backends.conf exists and Nginx was reloaded. |
| 403 Forbidden on a backend                   | Nginx cannot read the folder. Run <code>sudo chmod -R 755 /var/www/phase*</code> and make sure index.html exists.                                                   |
| Address already in use (port 80)             | Another service (Apache, another Nginx) uses port 80. Stop it, or change the load balancer to listen on 8080 and run <code>ngrok http 8080</code> .                 |
| Phases seem to skip (1, 3, 2...)             | Other visitors or favicon requests are using rotation turns. Keep the favicon rule from Step 3, test alone, and remember that all visitors share one rotation.      |
| Same page after every refresh                | The page is cached. Confirm the Cache-Control header is present ( <code>curl -sI http://localhost</code> ) and try a hard refresh (Ctrl+Shift+R).                   |
| Ngrok: authentication failed                 | The authtoken is missing or wrong. Re-run the add-authtoken command with the token from your dashboard.                                                             |
| Ngrok URL loads a warning page               | Expected on free accounts. Click Visit Site, or send the ngrok-skip-browser-warning header from scripts.                                                            |
| Ngrok URL shows offline / not found          | The tunnel stopped or the URL changed after a restart. Check the terminal for the current Forwarding URL.                                                           |
| Ngrok works but shows 502 from Ngrok         | Nothing is listening on the port you tunnelled. Run <code>curl -I http://localhost</code> and tunnel the port that responds.                                        |

Useful log commands:

```
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log
sudo systemctl status nginx
```

## 9. Cleanup and security

- **Stop the tunnel** with Ctrl+C in the Ngrok terminal when you finish. The public link stops working immediately.
- The public link exposes only the static Phase pages, but anyone who has the URL can open it. Do not tunnel a machine with sensitive services on port 80.
- Never share your Ngrok authtoken in screenshots, chats or public repositories. If it leaks, regenerate it in the dashboard.
- The backends bind to 127.0.0.1, so they are not reachable from the network directly.

To remove everything created in this guide:

```
sudo rm /etc/nginx/conf.d/phase-backends.conf /etc/nginx/conf.d/load-balancer.conf
sudo rm -rf /var/www/phase1 /var/www/phase2 /var/www/phase3
sudo nginx -t && sudo systemctl reload nginx
```

## Appendix A: Docker Compose alternative

Use this route on Windows or macOS (with Docker Desktop) or on any Linux machine with Docker. It builds the same architecture: three Nginx containers as Phase 1, 2, 3 and a fourth Nginx container as the load balancer. Ngrok still runs on the host.

### A.1 Folder and pages

```
mkdir phase-lb && cd phase-lb
mkdir phase1 phase2 phase3
COLORS=("1e88e5" "43a047" "e53935")
for i in 1 2 3; do
  C=${COLORS[$((i-1))]}
  cat > phase$i/index.html <<EOF
<!DOCTYPE html>
<html lang="en"><head><meta charset="utf-8"><title>Phase $i</title>
<style>body{margin:0;height:100vh;display:flex;align-items:center;
justify-content:center;font-family:Arial,sans-serif;background:$C;
color:#fff;text-align:center}h1{font-size:7rem;margin:0}</style></head>
<body><div><h1>PHASE $i</h1><p>Served by web server $i</p></div></body></html>
EOF
done
```

On Windows PowerShell, simply create the three folders and index.html files by hand with the same content, changing the number and colour.

### A.2 nginx.conf

```
upstream phase_servers {
    server phase1:80;
    server phase2:80;
    server phase3:80;
}
server {
    listen 80;
    location = /favicon.ico { access_log off; return 204; }
    location / {
        proxy_pass http://phase_servers;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        add_header Cache-Control "no-store, no-cache, must-revalidate" always;
    }
}
```

### A.3 docker-compose.yml

```
services:
  phase1:
    image: nginx:alpine
    volumes: ["/usr/share/nginx/html:ro"]
  phase2:
    image: nginx:alpine
    volumes: ["/usr/share/nginx/html:ro"]
  phase3:
    image: nginx:alpine
    volumes: ["/usr/share/nginx/html:ro"]
  lb:
    image: nginx:alpine
    ports: ["8080:80"]
    volumes: ["/etc/nginx/conf.d/default.conf:ro"]
    depends_on: [phase1, phase2, phase3]
```

### A.4 Run, test and publish

```
docker compose up -d
for i in $(seq 1 6); do curl -s http://localhost:8080 | grep -o "PHASE [0-9]"; done
ngrok http 8080
```

The load balancer is published on host port 8080, so Ngrok tunnels port **8080** here. Stop everything with `docker compose down`.

**Done**

You now have one public Ngrok address that rotates between Phase 1, Phase 2 and Phase 3 on every refresh.